

Question 10

Supermarket Checkout and Stock Consistency

Name	Registration No.
Aluvi Ian	C026-01-1064/2021
Kembero Collins Nyarumi	C026-01-0964/2022
Karanja Cyrus Njonge	C026-01-2610/2025

Scenario

A supermarket has several checkout counters, with each counter running as its own coroutine. Since different customers can buy the same product at the same time, the system makes sure the stock never goes below zero, even when the checkouts run in an interleaved way.

Design Overview

The supermarket has one shared stock table controlled by the scheduler. Checkout coroutines do not change the stock directly. Each checkout scans one item, uses `coroutine.yield()` to send the request to the scheduler, and waits for the scheduler's decision. Since Lua coroutines are cooperative, only one runs at a time, so the stock check and update cannot be interrupted.

(a) Checkout as a coroutine that yields after every scan — 4 marks

Each checkout is created using `coroutine.create()` and loops through its shopping list. After scanning an item, it uses `coroutine.yield(item.product, item.qty)` to send the request to the scheduler. The coroutine remembers where it stopped and continues from there when resumed.

(b) Shared stock table in the scheduler — 4 marks

The stock table is created in the scheduler and is not directly changed by the checkout coroutines. This gives the program one shared and correct record of the available stock.

(c) Checking stock before updating — 4 marks

`checkStock(product, qty)` is the only function that changes the stock. It first checks if `stock[product] >= qty`. If there is enough stock, it subtracts the quantity and returns success. If there is not enough, it rejects the request and leaves the stock unchanged. Since the check and update happen together, another checkout cannot interrupt them.

(d) Interleaved execution with consistent stock — 4 marks

The scheduler resumes the checkout coroutines one by one in round-robin order. This allows all checkouts to make progress at the same time instead of one finishing before another starts. If several checkouts request the

same product, the scheduler processes each request and rejects one when the stock is no longer enough. Therefore, the stock never becomes negative.

(e) Why single-threaded coroutines simplify stock control — 4 marks

In multithreading, two threads can access and change the same stock at the same time, causing a race condition. This usually requires locks or other synchronization methods. With Lua coroutines, switching only happens when `coroutine.yield()` is called. Since there is no `yield()` inside `checkStock()`, the check and update happen as one uninterrupted operation, keeping the stock consistent.

Full Lua Solution

```
-- =====
-- CCS 2105 - Question 10
-- Supermarket Checkout and Stock Consistency
-- =====
-- Design idea:
-- * Each checkout counter is its own coroutine. It never touches
--   the stock table directly - it only *asks* for permission.
-- * The shared stock table lives in the scheduler's scope, so
--   there is exactly one authoritative copy of the truth.
-- * The scheduler resumes checkouts in round-robin order. Because
--   only one coroutine can ever be running at a time, the
--   check-then-update sequence in checkStock() can never be
--   interrupted midway by another checkout - this is what keeps
--   stock logically consistent without locks.
-- =====

-- (b) Shared stock table - single source of truth, owned by the scheduler
local stock = {
    milk      = 12,
    bread     = 8,
    rice      = 15,
    sugar     = 6,
    cooking_oil = 4
}

-- Each checkout's shopping list: a sequence of {product, qty} requests
local checkoutData = {
    {
        name = "Checkout-1 (Amina)",
        cart = {
            { product = "milk", qty = 3 },
            { product = "bread", qty = 2 },
            { product = "sugar", qty = 5 }, -- will be contested / may fail
        }
    },
    {
        name = "Checkout-2 (Otieno)",
        cart = {
            { product = "milk", qty = 4 },
            { product = "rice", qty = 6 },
            { product = "cooking_oil", qty = 2 },
        }
    },
    {
        name = "Checkout-3 (Wanjiru)",
        cart = {
            { product = "milk", qty = 6 }, -- contests milk with 1 & 2
            { product = "sugar", qty = 2 },
            { product = "bread", qty = 3 },
        }
    }
}

-- (a) Each checkout is a coroutine that scans one product at a time and
--   yields the request out to the scheduler; it waits for the
--   scheduler's verdict before printing a receipt line and moving on.
local function checkoutProcess(name, cart)
    return coroutine.create(function()
        for _, item in ipairs(cart) do
            local approved, remaining = coroutine.yield(item.product, item.qty)
        end
    end)
end
```

```

        if approved then
            print(string.format(
                "[%s] Scanned %d x %-11s -> APPROVED (stock left: %d)",
                name, item.qty, item.product, remaining))
        else
            print(string.format(
                "[%s] Scanned %d x %-11s -> REJECTED (only %d in stock)",
                name, item.qty, item.product, remaining))
        end
    end
    return "receipt closed for " .. name
end)
end

-- (c) checkStock() is the single gatekeeper. It decides BEFORE any
--     update whether a purchase can proceed, then performs the update
--     as one atomic step (no coroutine can pre-empt this function).
local function checkStock(product, qty)
    local available = stock[product] or 0
    if available >= qty then
        stock[product] = available - qty
        return true, stock[product]
    else
        return false, available
    end
end

-- Build the coroutine list
local checkouts = {}
for _, c in ipairs(checkoutData) do
    table.insert(checkouts, { name = c.name, co = checkoutProcess(c.name, c.cart) })
end

print("=== Opening stock ===")
for _, p in ipairs({"milk", "bread", "rice", "sugar", "cooking_oil"}) do
    print(string.format(" %-11s : %d", p, stock[p]))
end
print()
print("=== Interleaved checkout processing ===")

-- (d) Round-robin scheduler demonstrating interleaved execution.
--     Each pass gives every live checkout exactly one turn, so all
--     three counters progress "quasi-simultaneously" while stock
--     stays consistent, since real updates only ever happen one at a
--     time inside checkStock().
local pendingArgs = {} -- results to feed back into each coroutine on its next resume
local active = #checkouts

while active > 0 do
    for _, entry in ipairs(checkouts) do
        if coroutine.status(entry.co) ~= "dead" then
            local args = pendingArgs[entry.co] or {}
            local ok, a, b = coroutine.resume(entry.co, table.unpack(args))

            if not ok then
                -- Error handling: never let one bad checkout crash the whole system
                print(string.format("[%s] ERROR: %s", entry.name, tostring(a)))
                active = active - 1
            elseif coroutine.status(entry.co) == "dead" then
                print(string.format("[%s] %s", entry.name, tostring(a)))
                active = active - 1
            else
                local product, qty = a, b

```

```

        local approved, remaining = checkStock(product, qty)
        pendingArgs[entry.co] = { approved, remaining }
    end
end
end

print()
print("=== Closing stock ===")
local negative_found = false
for _, p in ipairs({ "milk", "bread", "rice", "sugar", "cooking_oil" }) do
    print(string.format(" %-11s : %d", p, stock[p]))
    if stock[p] < 0 then negative_found = true end
end

print()
if negative_found then
    print("CONSISTENCY CHECK: FAILED - stock went negative!")
else
    print("CONSISTENCY CHECK: PASSED - no product ever went negative.")
end

```

Sample Terminal Output

Produced by running the script above.

```

=== Opening stock ===
milk      : 12
bread     : 8
rice      : 15
sugar     : 6
cooking_oil : 4

=== Interleaved checkout processing ===
[Checkout-1 (Amina)] Scanned 3 x milk      -> APPROVED (stock left: 9)
[Checkout-2 (Otieno)] Scanned 4 x milk     -> APPROVED (stock left: 5)
[Checkout-3 (Wanjiru)] Scanned 6 x milk    -> REJECTED (only 5 in stock)
[Checkout-1 (Amina)] Scanned 2 x bread     -> APPROVED (stock left: 6)
[Checkout-2 (Otieno)] Scanned 6 x rice     -> APPROVED (stock left: 9)
[Checkout-3 (Wanjiru)] Scanned 2 x sugar    -> APPROVED (stock left: 4)
[Checkout-1 (Amina)] Scanned 5 x sugar     -> REJECTED (only 4 in stock)
[Checkout-1 (Amina)] receipt closed for Checkout-1 (Amina)
[Checkout-2 (Otieno)] Scanned 2 x cooking_oil -> APPROVED (stock left: 2)
[Checkout-2 (Otieno)] receipt closed for Checkout-2 (Otieno)
[Checkout-3 (Wanjiru)] Scanned 3 x bread    -> APPROVED (stock left: 3)
[Checkout-3 (Wanjiru)] receipt closed for Checkout-3 (Wanjiru)

=== Closing stock ===
milk      : 5
bread     : 3
rice      : 9
sugar     : 4
cooking_oil : 2

CONSISTENCY CHECK: PASSED - no product ever went negative.

```